

Executive Summary

Designing a configurable ETL scheduler in Spring Boot involves several components. We propose storing job definitions (with fields like name, cron expression, enabled flag, etc.) in PostgreSQL and a separate table to log each job run's status (SUCCESS/FAILED). The Spring app exposes REST APIs to create/update/delete schedules and query job histories. Internally, a scheduling mechanism (e.g. Spring's `@Scheduled` or Quartz) regularly polls due jobs and dispatches them to an ETL execution service, which atomically records run start/end and status in the database. We compare options: Spring's simple scheduler (easy but no built-in clustering) vs. Quartz (rich features, DB-backed, cluster support [17†L164-L172]) vs. alternatives like ShedLock (using DB for locks [19†L240-L248]) or Postgres `pg_cron` (in-DB scheduling [47†L49-L57] [47†L61-L63]). Concurrency is handled via leader-election or database locks to prevent duplicate runs on multiple nodes [9†L363-L371] [15†L11-L20]. We design the schema (tables, indexes, constraints) and provide sample SQL migrations. APIs follow REST conventions (e.g. `POST /api/schedules`, `GET /api/schedules/{id}/runs`). For consistency, job status updates are done in transactions and jobs should be idempotent [23†L79-L83]. We outline error/retry strategies (e.g. Spring Retry or Quartz misfire settings), logging/metrics (job audit table, SLF4J/Micrometer), sample Spring Boot code (entities, repos, services, scheduler config, controllers), integration test examples, deployment notes (Flyway migrations, multi-node considerations), and security (API auth, input validation). The following sections detail each aspect comprehensively.

Database Schema and Migrations

Design two main tables: one for job **schedules** and one for **run history**. For example, a generic `schedule` table can include: job ID, name, cron expression, enable flag, and timestamps [4†L568-L576] [4†L581-L588]. Here is a sample Flyway migration snippet (PostgreSQL syntax):

```
-- V1__create_jobs_schema.sql
CREATE TABLE schedule (
  id SERIAL PRIMARY KEY,
  name VARCHAR(64) NOT NULL,
  description TEXT,
  cron_expr VARCHAR(64) NOT NULL,
  enabled BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
  start_date TIMESTAMP,
  end_date TIMESTAMP,
  last_triggered_at TIMESTAMP
);
CREATE INDEX idx_schedule_enabled ON schedule(enabled);
```

Each row defines a job's frequency (`cron_expr`), active window (`start_date` / `end_date`), etc. We enforce constraints like `PRIMARY KEY` and index the `enabled` flag for quick lookups of active jobs.

A `job_run` table logs each execution:

```
CREATE TABLE job_run (  
  id SERIAL PRIMARY KEY,  
  schedule_id INT NOT NULL REFERENCES schedule(id) ON DELETE CASCADE,  
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  ended_at TIMESTAMPTZ,  
  status VARCHAR(10) NOT NULL CHECK (status IN  
( 'SUCCESS', 'FAILED', 'RUNNING' )),  
  message TEXT  
);  
CREATE INDEX idx_job_run_sched_time ON job_run(schedule_id, started_at);
```

This table records when a job started/ended, its final status, and any messages or error output. Foreign-key and checks ensure referential integrity and valid status. A migration tool like Flyway or Liquibase would apply these SQL scripts at deployment.

REST API Design

The application exposes CRUD REST endpoints for managing schedules and querying runs, e.g.:

- **POST** `/api/schedules` – create a new job schedule. *Request JSON:*

```
{  
  "name": "Daily ETL Job",  
  "cronExpr": "0 0 2 * * ?",  
  "timezone": "UTC",  
  "startDate": "2026-01-01T00:00:00Z",  
  "enabled": true  
}
```

- **PUT** `/api/schedules/{id}` – update an existing schedule (e.g. change frequency or enable/disable).
- **DELETE** `/api/schedules/{id}` – delete a schedule.
- **GET** `/api/schedules` – list all schedules (with fields like id, name, cronExpr, enabled).
- **GET** `/api/schedules/{id}` – get details of one schedule.
- **GET** `/api/schedules/{id}/runs` – list past runs for job `{id}` (with status).
- **GET** `/api/schedules/{id}/runs/{runId}` – get details of a specific run (start/end times, status, logs).

Responses are JSON, e.g.:

```
GET /api/schedules/5/runs
```

```
HTTP 200 OK
```

```
[  
  {  
    "runId": 42,
```

```

    "startedAt": "2026-07-01T02:00:01Z",
    "endedAt": "2026-07-01T02:05:30Z",
    "status": "SUCCESS"
  },
  {
    "runId": 41,
    "startedAt": "2026-06-30T02:00:02Z",
    "endedAt": "2026-06-30T02:06:10Z",
    "status": "FAILED",
    "message": "Database timeout"
  }
]

```

These endpoints use typical Spring MVC (`@RestController`, `@RequestMapping`) and JPA repositories for DB operations. Input validation (e.g. checking cron syntax, required fields) and security (authentication/authorization) must be applied to these APIs.

Scheduling Mechanisms (Spring Scheduler vs Quartz vs Others)

We compare common options in a summary table:

Mechanism	Persistence/ State	Clustering Support	Pros	Cons
Spring @Scheduled	In-memory (stateless)	X (no built-in)	Simple to use (annotate methods); light-weight. Easy fixed-rate or cron tasks.	No built-in persistence or distribution. All nodes would execute tasks in HA setups 【17†L229-L235】 . Limited to simple schedules (fixed delay/rate or cron).
Quartz Scheduler	DB-backed (JDBC JobStore)	✓ (with clustering)	Advanced features: persistent cron jobs, transactions, clustering/HA 【17†L164-L172】 【15†L11-L20】 . Supports misfire policies, job recovery.	More complex setup: requires additional tables (from Quartz schema), config, and Spring integration. Heavier framework.
ShedLock (with @Scheduled)	Lock in DB/ Redis	✓ (via lock)	Lets you use Spring's annotations but with a shared lock. Simple DB table or Redis to ensure only one node runs a task at a time 【19†L240-L248】 .	Added library; tasks still defined in code (no persistent job store). Slight overhead for lock table.

Mechanism	Persistence/ State	Clustering Support	Pros	Cons
Kubernetes CronJob / External	In-cluster/ External	✓ (in Kubernetes)	Offloads scheduling to Kubernetes or another orchestrator. Good for containerized environments.	Less flexible for job-config in DB; outside Spring Boot's control. Not covered in detail here.
PostgreSQL pg_cron/ pgAgent	DB-level scheduling	Limited (DB extension)	Native to Postgres: schedule SQL or scripts directly in DB 【47†L49-L57】 【47†L61-L63】 . No external app needed once set up.	Requires Postgres extension and superuser rights. Limited to SQL-level jobs, not general Java ETL. Not integrated with Spring.

Spring's `TaskScheduler` (used by `@Scheduled`) is easy to enable (just add `@EnableScheduling`), but it is purely local. As one commenter noted, Quartz "supports persistent, transactional and distributed jobs" but is "more complex" than Spring's built-in scheduler [【17†L164-L172】](#) . Indeed, by default Spring will run the same `@Scheduled` task on *every* instance, leading to duplicates in a cluster [【19†L240-L248】](#) . Quartz solves this via a JDBCJobStore: all nodes share a DB, and only one "wins" a lock to fire each job [【15†L11-L20】](#) . For high-availability, Quartz auto-recovers jobs marked for recovery if a node fails [【15†L32-L39】](#) .

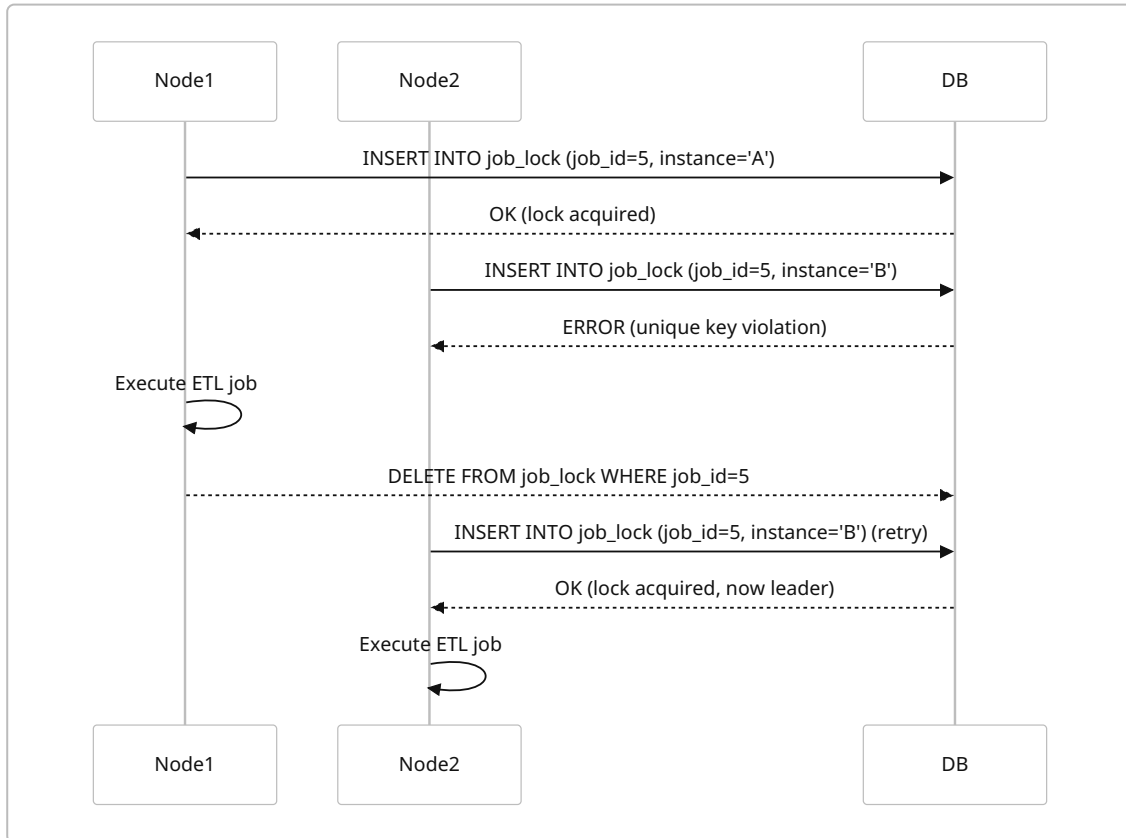
Alternatively, ShedLock can be used to wrap a `@Scheduled` job so that before executing it acquires a row-lock (in a "shedlock" table). This ensures only one instance proceeds, with minimal config [【19†L246-L248】](#) . Quartz is heavier but out-of-the-box supports misfire policies and clustering. Which to choose depends on needs: **@Scheduled** is best for simple, single-node apps; **Quartz** for rich features and true clustering [【17†L229-L235】](#) ; **ShedLock** for a middle-ground in a multi-instance Spring app. Kubernetes CronJobs or Postgres' own schedulers (pg_cron) are alternatives outside the Spring scope.

Concurrency & Distributed Execution

In a clustered deployment, we must prevent multiple nodes from running the same job simultaneously. Quartz's JDBCJobStore handles this with built-in locking: "when a trigger's firing time occurs, the first node to acquire it (by placing a lock on it) is the node that will fire it... it won't necessarily be the same node each time" [【15†L19-L27】](#) . If a node crashes mid-job, remaining nodes "detect the condition and identify the jobs... Jobs marked for recovery... will be re-executed" [【15†L32-L39】](#) . This provides automatic failover and load-balancing. Spring Boot's Quartz support auto-configures a `SchedulerFactoryBean` when you include `spring-boot-starter-quartz` [【49†L1-L4】](#) . You must set `org.quartz.jobStore.isClustered=true` and give each node a unique `instanceId=AUTO` (as per Quartz docs [【15†L48-L56】](#)). Quartz requires clock sync across nodes (e.g. NTP) to avoid misfires [【15†L58-L66】](#) .

If not using Quartz, one can implement a leader-election scheme. For example, a simple approach is a DB "locks" table or "nodes" table where each instance registers itself and elects a leader. Only the leader runs scheduled tasks [【9†L363-L371】](#) . In one implementation, each pod writes to a `SYS_NODE` table with a timestamp; a scheduled leader-election job sets `isLeader=true` for one node, and the scheduled task checks `if (currentNode.isLeader())` before proceeding [【9†L373-L382】](#)

【9†L385-L394】. A PostgreSQL-specific lock table is even simpler: use a row per job with a `UNIQUE (job_id)` constraint; before running, do `INSERT INTO job_lock(job_id, instance_id, locked_at) VALUES (...)`. One insert will succeed, others get a unique-constraint error and skip. On completion, delete the lock row. This ensures only one node holds the lock. The diagram below illustrates this locking sequence:



In summary, ensure *single execution* via one of these methods: Quartz clustering (easy, built-in) or manual locks/leader election (extra code). ShedLock (not shown above) abstracts the lock with annotations. Without such measures, `@Scheduled` tasks will run on all nodes 【17†L229-L235】.

Transactionality & Consistency

Each job run's status updates must be atomic and recoverable. In practice, the ETL task execution should bracket database writes in a transaction and update the job status as part of that. A typical pattern: when a job starts, insert a `job_run` row with `status='RUNNING'` (in a transaction). After all ETL work completes successfully, update that row to `status='SUCCESS'` and set `ended_at`. If an exception occurs, catch it (or rely on a global exception handler) and update `status='FAILED'` in a separate transaction. This ensures no run is left in unknown state. Using Spring's `@Transactional`, one can do:

```

@Transactional
public JobRun runJob(Long scheduleId) {
    JobRun run = jobRunRepo.save(new JobRun(scheduleId, now(), null,
"RUNNING", null));
    try {

```

```

        // ... perform ETL steps here ...
        run.setEndedAt(LocalDateDateTime.now());
        run.setStatus("SUCCESS");
    } catch (Exception e) {
        run.setEndedAt(LocalDateDateTime.now());
        run.setStatus("FAILED");
        run.setMessage(e.getMessage());
    }
    return jobRunRepo.save(run);
}

```

If ETL steps involve multiple DB updates, they should be in their own transactions or use Spring Batch for chunk-oriented processing. Always update the status row after those commits. Also consider *idempotency*: ETL jobs should be written so that re-running them (e.g. after a crash and restart) does not corrupt data. Use UPSERT/MERGE logic or clear staging tables first. As one expert notes, “Without idempotence, retries or duplicate executions could result in... duplicate data or inconsistent states. By designing idempotent jobs, you make your system resilient” [23†L79-L83] . In practice, generate idempotency keys or use SQL MERGE for inserts.

For concurrency, a transaction isolation level of `SERIALIZABLE` is usually unnecessary; `READ COMMITTED` with explicit row-locks (`SELECT ... FOR UPDATE`) on the `schedule` table can prevent two threads from picking up the same job. Alternatively, a single poller thread with non-overlapping triggers obviates this. Use optimistic locking (`@Version` column) if you want to avoid lost updates to metadata like `last_triggered_at` .

Error Handling and Retry Policies

Jobs will inevitably fail (e.g. network issues, invalid data). We should distinguish transient vs permanent errors. For transient failures (e.g. timeout), consider automatic retries with exponential backoff. Spring Retry provides `@Retryable` or a `RetryTemplate` to re-invoke a failed operation [44†L219-L227] . For example, annotate the ETL method with `@Retryable(maxAttempts=3, backoff=@Backoff(delay=5000))` so it tries again on exception. If all retries fail, mark `status=FAILED` and log the stack trace (avoid swallowing exceptions). Note: do **not** catch and suppress exceptions too broadly, as that prevents retry logic [23†L79-L83] .

Quartz has its own retry (misfire) settings: if a trigger was missed, Quartz can fire it immediately or skip based on `misfireThreshold` and `misfireInstruction` . We should configure sensible defaults (e.g. skip if too late).

In any case, record failures clearly (in logs and `job_run.message`) and possibly alert an operator. Long-running jobs might need periodic heartbeats or interim commits so that partial progress is not lost. Define a failure policy: e.g. after N consecutive failures, disable the schedule or trigger a warning. In summary: *automate retries for known transient issues, and make failures visible* (through status, logs, and metrics).

Audit, Logging, and Metrics

Maintain an audit trail by logging key events and writing to the `job_run` table. Each run entry is inherently an audit record (with timestamps, status, error message). Additionally, log to a centralized

system (SLF4J) at INFO/WARN levels: when a job starts, ends, or fails. Use [MDC logging](#) to tag logs with `jobId` or `runId`. Consider AOP or decorators to log method entry/exit and durations (tools like JAMon or custom interceptors [\[35†L157-L165\]](#)).

For metrics, integrate Spring Boot Actuator/Micrometer: define counters (total runs, successes, failures) and timers (execution time). Example: a counter per job name, or a Prometheus gauge of “last run status.” Expose these for monitoring dashboards. Store the number of rows processed or execution time in the `message` or a dedicated column if needed.

Audit examples: If regulations require it, also log who created/modified schedules (if multi-user) by adding `created_by` fields. But at minimum, ensure each run is timestamped and its outcome saved. Enable Actuator’s `/metrics` to get JVM-level stats, and define custom metrics via `@Timed` or `MeterRegistry`.

Sample Spring Boot Code Snippets

Below are illustrative code excerpts.

JPA Entities

```
@Entity
@Table(name="schedule")
public class JobSchedule {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private String cronExpr;
    private boolean enabled = true;
    private String timezone;
    private LocalDateTime startDate;
    private LocalDateTime endDate;
    // ... getters/setters ...
}

@Entity
@Table(name="job_run")
public class JobRun {
    @Id @GeneratedValue
    private Long id;
    @ManyToOne @JoinColumn(name="schedule_id")
    private JobSchedule schedule;
    private LocalDateTime startedAt;
    private LocalDateTime endedAt;
    private String status; // e.g. "RUNNING", "SUCCESS", "FAILED"
    private String message;
    // ... getters/setters ...
}
```

Repositories

```
public interface JobScheduleRepository extends
JpaRepository<JobSchedule, Long> {
    List<JobSchedule> findByEnabledTrueAndStartDateBeforeAnd(
        LocalDateTime now);
}
public interface JobRunRepository extends JpaRepository<JobRun, Long> { }
```

Use Spring Data JPA (Hibernate) to handle CRUD. The repository can have a custom query like

```
@Query("SELECT j FROM JobSchedule j WHERE j.enabled = true AND (j.startDate
IS NULL OR j.startDate <= :now) AND (j.endDate IS NULL OR j.endDate
>= :now)")
List<JobSchedule> findActiveSchedules(@Param("now") LocalDateTime now);
```

to fetch due jobs.

Scheduling Service

Using Spring's `@Scheduled` to poll the DB:

```
@Service
public class SchedulePoller {
    @Autowired JobScheduleRepository scheduleRepo;
    @Autowired JobRunnerService runner;

    @Scheduled(cron = "0 * * * * *") // every minute
    public void pollSchedules() {
        List<JobSchedule> jobs =
            scheduleRepo.findActiveSchedules(LocalDateTime.now());
        for (JobSchedule job : jobs) {
            runner.executeJob(job);
        }
    }
}
```

Alternatively, configure a `Trigger` bean if dynamic schedules are needed. For Quartz, define `JobDetail` and `CronTrigger` beans.

Job Execution Service (Transactional)

```
@Service
public class JobRunnerService {
    @Autowired JobRunRepository runRepo;

    @Transactional
```

```

public void executeJob(JobSchedule schedule) {
    // Acquire lock (if using manual locking strategy)
    // ...

    JobRun run = new JobRun();
    run.setSchedule(schedule);
    run.setStartedAt(LocalDateDateTime.now());
    run.setStatus("RUNNING");
    runRepo.save(run);

    try {
        // Perform ETL logic here (could be another service call)
        performEtlWork(schedule);

        run.setEndedAt(LocalDateDateTime.now());
        run.setStatus("SUCCESS");
    } catch (Exception ex) {
        run.setEndedAt(LocalDateDateTime.now());
        run.setStatus("FAILED");
        run.setMessage(ex.getMessage());
        // (Optional) log exception stack trace
    } finally {
        runRepo.save(run);
    }
}
}

```

Transaction ensures the status row is updated atomically. If concurrency control is needed, annotate with an exclusive `@Transactional(propagation=REQUIRES_NEW, isolation=Isolation.SERIALIZABLE)` or use `SELECT ... FOR UPDATE` on the schedule row before proceeding.

Controller Example

```

@RestController
@RequestMapping("/api/schedules")
public class ScheduleController {
    @Autowired private JobScheduleRepository repo;
    @Autowired private JobRunRepository runRepo;

    @PostMapping
    public ResponseEntity<?> create(@RequestBody JobSchedule dto) {
        JobSchedule saved = repo.save(dto);
        return ResponseEntity.ok(saved);
    }

    @GetMapping("/{id}/runs")
    public List<JobRun> getRuns(@PathVariable Long id) {
        return runRepo.findByScheduleId(id);
    }
}

```

```
// ... other endpoints ...  
}
```

Integration Testing & SQL Test Data

Use Spring's testing support. For example, an integration test with an in-memory H2 database or Testcontainers Postgres:

```
@SpringBootTest  
@AutoConfigureMockMvc  
@ActiveProfiles("test")  
public class ScheduleApiTests {  
    @Autowired MockMvc mockMvc;  
  
    @BeforeEach  
    public void setup() {  
        // Insert test data (via repository or @Sql)  
    }  
  
    @Test  
    public void testCreateAndQuerySchedule() throws Exception {  
        String json = "{\"name\":\"Test Job\",\"cronExpr\":\"*/5 * * * *  
*\"}";  
        mockMvc.perform(post("/api/schedules")  
            .contentType(MediaType.APPLICATION_JSON)  
            .content(json)  
            .andExpect(status().isOk())  
            .andExpect(jsonPath("$.id").isNumber());  
  
        mockMvc.perform(get("/api/schedules"))  
            .andExpect(status().isOk())  
            .andExpect(jsonPath("$[0].name").value("Test Job"));  
    }  
}
```

Use `@Sql` or `repo.save()` to seed data for tests. Also test scheduling behavior by mocking time or manually invoking the polling service, then checking that `job_run` records are created. For DB integration, use Flyway in tests to apply the same migration scripts.

Deployment Considerations & Runbook

- **Migrations:** Automate schema creation (Flyway/Liquibase) on startup or CI/CD. Include steps in runbook to verify tables exist.
- **Configuration:** Parameterize DB URL, credentials, and scheduling thread pool via Spring `application.yml` (e.g. `spring.datasource.*`, `spring.quartz.properties.*` if using Quartz). If using Quartz, ensure the Quartz tables (from its schema SQL) are created beforehand or auto-generated.
- **Clustering:** For multi-node deployments, decide on approach: run a single instance for scheduling (simplest), or multiple with Quartz clustering/ShedLock. If using Kubernetes, you may

use a single Pod or a Deployment with leader election (Spring Cloud Kubernetes) to ensure one active scheduler. Document how to add a new instance to the cluster (e.g. DB lock table entry).

- **Monitoring:** Configure logging (to file/ELK) and metrics export (Prometheus/Grafana). The runbook should include how to check job statuses (e.g. query `job_run` table or view a dashboard) and how to reschedule failed jobs if needed.
- **Restart:** On application restart, ensure running jobs are idempotent; the scheduler should resume polling. If using Quartz, it will pick up persistent jobs automatically. The runbook should note how to restart gracefully (`SIGTERM`), as Spring will complete running tasks or wait for them to finish.
- **Failover:** Document how failover works: if a node crashes, locks should be released after TTL or via manual cleanup. When using leader election, note how to handle split-brain or manual leader override.

Security Considerations

- **Authentication/Authorization:** Protect the REST API (e.g. OAuth2/JWT or basic auth). Restrict schedule management to authorized roles (e.g. ADMIN).
- **Input Validation:** Sanitize and validate the cron expression (to avoid injection attacks) and other fields. Reject invalid schedules or excessively frequent ones.
- **Principle of Least Privilege:** The database user should have just the needed rights (SELECT, INSERT, UPDATE on the scheduler schema). Encrypt sensitive config (DB passwords) and use TLS for DB connections if possible.
- **Multi-Tenancy (optional):** If serving multiple tenants, include a `tenant_id` in the `schedule` table and filter queries accordingly. Never let one tenant's schedule trigger jobs for another. Implement row-level security if needed.
- **Data Protection:** If ETL jobs handle sensitive data, ensure logs/messaging don't leak secrets. Clear the `message` field of exceptions of sensitive info.
- **Concurrency:** Validate that only one scheduler instance can execute a job (to prevent duplicated data processing as a security/data integrity issue).

Overall, the system should fail safely: misfires or errors should not corrupt the database state. All job-triggering and status updates should be done with proper transactions (no partial commits). Periodically review security patches for Spring/Quartz and follow best practices (e.g. secure defaults in Spring Boot 3.x).

References: We base this design on patterns and documentation from Spring and Quartz communities [\[4†L568-L576\]](#) [\[17†L164-L172\]](#) [\[17†L229-L233\]](#) [\[19†L240-L248\]](#) [\[23†L79-L83\]](#) [\[15†L11-L20\]](#) [\[9†L363-L371\]](#) , adapting them to a PostgreSQL-backed Spring Boot application. The above schema and code snippets are illustrative adaptations of those recommendations.
